

Edge-Cloud Fall Detection with Quality-Aware Bidirectional Temporal Modeling

Yarong Huang, Jiahao Zhao, Chengyi Xu, Zhonglin Chang, and Jiayun Song
Faculty of Applied Sciences, Macao Polytechnic University, Macao SAR, China

Abstract—Camera-based fall detection requires reliable decisions from incomplete pose sequences and continued operation during network interruptions. Performance on complete sequences alone does not establish either capability. Here we introduce MaskedBiMamba, a skeleton sequence classifier that combines bidirectional state-space modeling, temporal attention, frame masking, and quality-weighted pooling. Across four subject-disjoint folds and three seeds on CAUCAFall and GMDCSA-24, it achieved the highest window precision among five temporal models. Under 30% random frame loss, its window F1 was 0.6276 ± 0.0507 , compared with 0.4361 ± 0.0669 for the temporal convolution–Transformer baseline. The Transformer encoder led clean window F1, while masking and quality pooling showed mixed clean-data effects in component ablations. We integrated the classifier with Hailo-accelerated pose extraction, local alert storage, and deferred cloud delivery on a Raspberry Pi. The disconnected video pipeline processed 20.56 frames/s, and recovery tests preserved all 85 evaluated records and their original capture times. These results support combining temporal classification with persistent edge execution, while continuous-monitoring accuracy and alert calibration require further evaluation.

Index Terms—fall detection, skeleton sequence, edge-cloud system, temporal attention, Mamba.

Code and Project Materials—

<https://github.com/EmpyreanHYR/MPU-IoT-Project1-Fall-Detection>

I. INTRODUCTION

Falls can resemble sitting, kneeling, or lying down when viewed in a single frame. Their distinction depends on the descent and subsequent posture. Camera-based monitoring captures this motion without a wearable device, but occlusion and tracking failures can interrupt pose observations. Skeleton sequences provide a compact motion representation, yet missing observations can alter the evidence available to a temporal classifier.

The intended setting is an indoor assisted-living space, where caregivers review suspected falls and acknowledge events through a monitoring interface. The design objectives are to process video near the camera, continue detection during network outages, and preserve event times when records reach the cloud after reconnection.

Temporal classifiers relate pose estimates across frames rather than treating each frame independently. Yu et al. [1] combined dilated temporal convolutions, a Transformer encoder, and weighted focal loss in TCNTE, with deployment on a Jetson device. Kim et al. [2] combined tracking, AlphaPose,

and CNN-LSTM classification in YOSAP-LSTM for construction scenes with occlusion. These systems connect temporal classification to the constraints of an edge processing pipeline.

Observation quality is also relevant to the temporal representation. Zhang et al. [3] combined video-audio fusion, temporal attention, and frame masking in Fall-Mamba. Chun et al. [4] incorporated skeleton geometry and confidence information into StoneGAT, with node-feature removal during training. Building on these approaches, we examine whether combining temporal context and pose quality preserves classification performance when extracted observations are interrupted.

We introduce MaskedBiMamba, combining independent forward and backward Mamba branches [5], temporal attention, training-time frame masking, and quality-weighted pooling. Both temporal directions operate within an already buffered one-second window. Matched comparisons, component ablations, and pose perturbations examine clean classification, frame-loss tolerance, and the balance between sensitivity and false predictions. Separate window, video, event, and confirmed-alert measures distinguish classifier outputs from operational alerts.

We integrate the classifier into FallGuard, an edge-cloud prototype with Hailo-accelerated pose extraction, local inference, persistent records, and an authenticated monitoring interface. Offline replay and cloud recovery tests evaluate whether detection continues during disconnection and whether records retain their identity and capture time after delivery.

II. MASKEDBIMAMBA FOR SKELETON-BASED FALL DETECTION

A. Pose representation and temporal windows

MaskedBiMamba receives a sequence of estimated poses of one person. We retain $J = 12$ body joints, excluding the five facial keypoints. Let $r_{t,j}^{(a)}$ be joint j 's coordinate along axis $a \in \{x, y\}$ at time t , and $c_{t,j} \in [0, 1]$ its confidence. Each axis is normalized within a frame:

$$\bar{r}_{t,j}^{(a)} = \frac{r_{t,j}^{(a)} - \min_k r_{t,k}^{(a)}}{\max(\max_k r_{t,k}^{(a)} - \min_k r_{t,k}^{(a)}, 10^{-4})}. \quad (1)$$

The frame vector $\mathbf{x}_t = [\bar{r}_{t,j}^{(x)}, \bar{r}_{t,j}^{(y)}, c_{t,j}]_{j=1}^J \in \mathbb{R}^{36}$ describes relative body configuration and joint confidence. Per-frame normalization reduces dependence on image position and body scale. A separate quality vector records the mean joint

confidence, visible-joint fraction, and person-box confidence b_t :

$$\mathbf{q}_t = \left[\frac{1}{J} \sum_{j=1}^J c_{t,j}, \frac{1}{J} \sum_{j=1}^J \mathbb{I}(c_{t,j} \geq 0.2), b_t \right]^\top. \quad (2)$$

Here $\mathbb{I}(\cdot)$ is the indicator function.

Each input window spans 1.0 s and is linearly interpolated to $T = 30$ time steps. Successive windows advance by 0.2 s.

B. Masked input and temporal attention

MaskedBiMamba combines frame masking, temporal attention, bidirectional Mamba stacks, and quality-weighted pooling (Fig. 1). The architecture relates observations before and after a posture change while retaining pose quality as a separate input. The selective state-space operator itself follows Mamba [5].

During training, a Bernoulli variable m_t determines whether the pose at time t is retained. The masked pose is embedded in $d = 64$ dimensions:

$$m_t \sim \text{Bernoulli}(1 - \rho), \quad \mathbf{e}_t = \mathbf{W}_e(m_t \mathbf{x}_t) + \mathbf{b}_e + \mathbf{p}_t, \quad (3)$$

where $\rho = 0.15$, $\mathbf{W}_e \in \mathbb{R}^{d \times 36}$, $\mathbf{b}_e \in \mathbb{R}^d$, and $\mathbf{p}_t \in \mathbb{R}^d$ is a learned position embedding. If every frame is masked, the first is retained. Training-time masking exposes the classifier to windows with missing frame observations. The quality stream remains unchanged; inference uses $m_t = 1$ for every frame.

Stacking $\mathbf{e}_t^\top$ gives $\mathbf{E} \in \mathbb{R}^{T \times d}$. Four attention heads compare all positions within the window. For head k , with $d_h = d/4 = 16$,

$$\mathbf{A}_k = \text{softmax} \left(\frac{(\mathbf{E} \mathbf{W}_k^Q)(\mathbf{E} \mathbf{W}_k^K)^\top}{\sqrt{d_h}} \right) (\mathbf{E} \mathbf{W}_k^V), \quad (4)$$

where the softmax acts along each row and $\mathbf{W}_k^Q, \mathbf{W}_k^K, \mathbf{W}_k^V \in \mathbb{R}^{d \times d_h}$ are learned projections. The attended sequence is

$$\mathbf{H} = \text{LN}(\mathbf{E} + [\mathbf{A}_1 \parallel \dots \parallel \mathbf{A}_4] \mathbf{W}^O), \quad (5)$$

where $\parallel$ denotes feature concatenation, $\mathbf{W}^O \in \mathbb{R}^{d \times d}$, and LN is layer normalization. Affine projection biases are omitted in Eqs. (4) and (5) for readability. Attention connects observations at different positions before the recurrent state updates.

C. Selective state space and bidirectional fusion

Mamba's selective state-space update makes its step size and input/output coefficients depend on the current feature [5]. We retain the standard Mamba input projection, causal depthwise convolution, SiLU gate, selective scan, and output projection. Each block uses state size 16, convolution width 4, and expansion factor 2, giving 128 internal channels. Our architectural change consists of independent forward and backward processing. The underlying Mamba operator is retained without a change to its selective update.

To use both temporal directions, independent forward and backward stacks process the same attended window. Let $\mathcal{R}$ reverse the time axis, and let $\mathcal{M}_f^{(\ell)}$ and $\mathcal{M}_b^{(\ell)}$ denote the

Mamba blocks at layer ℓ . Starting with $\mathbf{F}^{(0)} = \mathbf{H}$ and $\mathbf{B}^{(0)} = \mathcal{R}(\mathbf{H})$, the two residual stacks compute

$$\begin{aligned} \mathbf{F}^{(\ell)} &= \mathbf{F}^{(\ell-1)} + \mathcal{M}_f^{(\ell)}(\mathbf{F}^{(\ell-1)}), \\ \mathbf{B}^{(\ell)} &= \mathbf{B}^{(\ell-1)} + \mathcal{M}_b^{(\ell)}(\mathbf{B}^{(\ell-1)}), \end{aligned} \quad \ell = 1, 2. \quad (6)$$

The reverse output is restored to the original time order before fusion:

$$\mathbf{H}^* = \text{LN} \left([\mathbf{F}^{(2)} \parallel \mathcal{R}(\mathbf{B}^{(2)})] \mathbf{W}_f + \mathbf{b}_f \right), \quad (7)$$

where $\mathbf{W}_f \in \mathbb{R}^{128 \times 64}$ and $\mathbf{b}_f \in \mathbb{R}^{64}$. Its row $\mathbf{h}_t^\top$ describes time t using preceding and subsequent observations in the buffered window. The aligned representation relates a descent to its subsequent posture within the buffered window. Bidirectional processing produces one classification after that window has been collected.

D. Quality-weighted pooling and classification

Uniform pooling assigns the same coefficient to every frame, even when pose confidence varies. We instead learn a scalar weight from each quality vector:

$$a_t = \max \{ 10^{-6}, \sigma [\mathbf{w}_2^\top \text{ReLU}(\mathbf{W}_1 \mathbf{q}_t + \mathbf{b}_1) + b_2] \}, \quad (8)$$

where $\mathbf{W}_1 \in \mathbb{R}^{32 \times 3}$, $\mathbf{b}_1, \mathbf{w}_2 \in \mathbb{R}^{32}$, $b_2 \in \mathbb{R}$, and σ is the sigmoid function. The gate is trained jointly with the classifier. Normalizing its outputs gives

$$\alpha_t = \frac{a_t}{\sum_{u=1}^T a_u}, \quad \mathbf{z} = \sum_{t=1}^T \alpha_t \mathbf{h}_t. \quad (9)$$

The coefficients satisfy $\alpha_t > 0$ and $\sum_t \alpha_t = 1$. These coefficients adjust each frame's contribution using the separate quality input. Equal coefficients recover temporal mean pooling over the complete window. The window representation $\mathbf{z} \in \mathbb{R}^{64}$ yields two class logits and probabilities:

$$\ell = \mathbf{W}_c \mathbf{z} + \mathbf{b}_c, \quad \pi_c = \frac{e^{\ell_c}}{e^{\ell_0} + e^{\ell_1}}, \quad c \in \{0, 1\}, \quad (10)$$

where $\mathbf{W}_c \in \mathbb{R}^{2 \times 64}$, $\mathbf{b}_c \in \mathbb{R}^2$, and class 1 denotes a fall.

E. Training objective

The training set contains more non-fall than fall windows. Weighted focal loss [6] reduces the influence of easy examples while balancing the two classes:

$$\begin{aligned} \mathcal{L} &= -\frac{1}{N_b} \sum_{i=1}^{N_b} w_{y_i} (1 - \pi_{i,y_i})^\gamma \log \pi_{i,y_i}, \\ w_c &= \frac{N_{\text{train}}}{2N_c}. \end{aligned} \quad (11)$$

Here N_b is the batch size, y_i is the true class, π_{i,y_i} is its predicted probability, N_c is the training count of class c , and $N_{\text{train}} = N_0 + N_1$. The exponent γ controls the down-weighting of easy examples.

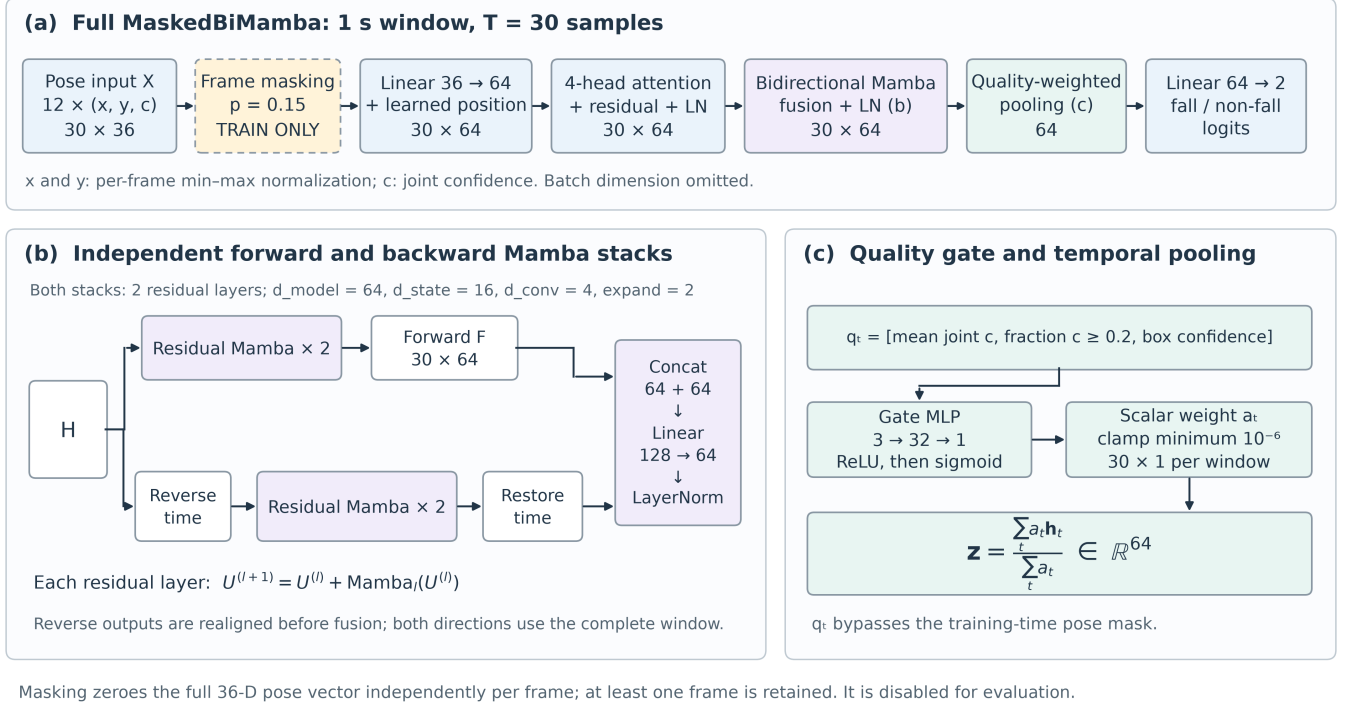


Figure 1: MaskedBiMamba architecture. Pose features pass through training-time frame masking, temporal attention and two independent Mamba stacks. The reverse stream is aligned before fusion. A separate quality network supplies the temporal pooling weights. Tensor dimensions omit the batch axis.

III. SYSTEM DESIGN

FallGuard performs pose extraction and temporal classification on the Raspberry Pi. The Hailo accelerator runs the pose network, while the Pi CPU evaluates a one-second skeleton window with MaskedBiMamba. The local alert state and prediction records remain available during network outages. After reconnection, a synchronization service transfers pending records to the cloud. The cloud retains these records for the event history and monitoring dashboard. Figure 2 shows this data flow.

Routine synchronization transfers pose keypoints, model scores, timestamps, and alert records, but not stored video. An authenticated preview can relay annotated images on demand; these frames remain in memory and are not added to the event archive. Pose records can reveal movement and are treated as protected application data. The absence of routine video transfer does not establish anonymity.

The deployed backend runs on a Tencent Cloud Lighthouse VM with OpenCloudOS 9. The Python service is bound to the VM loopback interface behind an Nginx HTTPS gateway. Dashboard access uses HTTP Basic authentication over TLS. Edge records are pulled through a Tailscale private route from a token-protected endpoint; the corresponding public device path is not exposed. Service credentials are loaded from permission-restricted environment files rather than stored in the submitted source. These controls describe the implemented

deployment boundary; they do not constitute a formal privacy or security evaluation.

A. Alert-state transitions

The edge node converts consecutive window probabilities into four alert states: NORMAL, SUSPECTED, CONFIRMED, and CLEARED states. Let p_k be the probability of the k th locally classified window. The high- and low-probability counters are

$$\begin{aligned} c_k^+ &= \mathbf{1}[p_k \geq 0.65](c_{k-1}^+ + 1), \\ c_k^- &= \mathbf{1}[p_k \leq 0.35](c_{k-1}^- + 1). \end{aligned} \quad (12)$$

A high observation opens a suspected event. Confirmation requires $c_k^+ \geq 2$ and $p_k \geq 0.82$; $c_k^- \geq 2$ clears an active event. An isolated high-probability observation is therefore insufficient to confirm an event. Delivery acknowledgement is separate from these algorithmic alert-state transitions.

B. Offline inference and hardware acceleration

The implementation reuses Ultralytics pose estimation [7], HailoRT, and Mamba operators [5]; TCNTE [1] is a comparison baseline. FallGuard integrates quality-aware bidirectional classification with local alert storage and deferred cloud synchronization.

The repository includes model/metric code, configurations, manuscripts, aggregate results, selected temporal weights, and deployment source with generic service templates. Credentials,

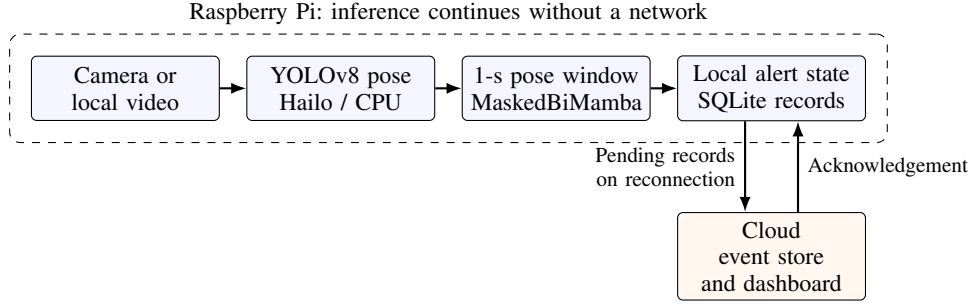


Figure 2: Local inference and deferred cloud synchronization. CPU pose estimation provides a fallback when the Hailo backend is unavailable. Cloud acknowledgement follows durable storage, allowing safe retries after an interrupted transfer.

raw videos, and runtime databases are excluded. Exact training replay requires reconstructing the absent original training manifests and pose caches.

The runtime reads a USB camera or local video. FFmpeg preserves video timestamps. Hailo runs YOLOv8s-pose, decodes 17 COCO keypoints, and selects the highest-confidence person; 12 body joints feed the temporal model. CPU ONNX provides a pose fallback.

A timestamped buffer forms one-second windows up to five times per second, interpolates to 30 samples, and applies training normalization. The quality input uses Eq. (2). ONNX Runtime executes MaskedBiMamba on the Pi CPU. Separate camera acquisition avoids accumulating stale frames.

C. Record storage and synchronization

SQLite commits each prediction and alert state together. The independent private outbox exposes pending records. Cloud import deduplicates identifiers and preserves observation times. Delivery is acknowledged only after the cloud transaction commits; retry drains pending batches after reconnection.

D. Detection interface

The local and cloud workspaces provide overview, history, and diagnostics. The Pi page reads existing preview/status/SQLite without a second inference pipeline. It includes a 120-window chart, state/time/delivery filters, pagination, CSV, and temperature/memory/disk/load readings, all accessible locally without networking.

The cloud queries its durable archive and reads current Pi status/preview through a token-protected private bridge. Invalid poses, expired predictions, or edge loss clear current probability; unavailable pending counts are unknown. Same-origin JSON and a page token protect controls. Pausing preserves history and the outbox.

The 7 October 2026 deployment check verified edge version 2026.10.07 and cloud version 2026.10.07-cloud. The update retained 3,304 windows and 306 event rows, with zero pending records: operational counts, not evaluation samples. Twenty-one temporary-fixture backend/integration tests and both actual page-script checks passed. They establish interface and delivery behavior, not accuracy. Figure 3 shows the deployed workspace.

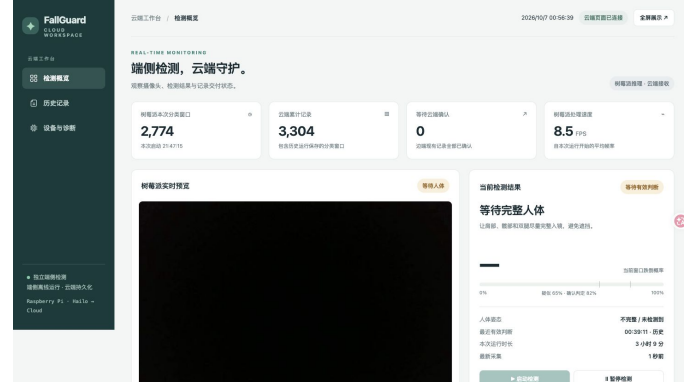


Figure 3: Deployed cloud workspace, 7 October 2026. Retained history and an authenticated Pi preview are visible; no complete person is present, so current probability is blank. This is operational evidence, not an accuracy result.

IV. EXPERIMENTS

A. Datasets and evaluation protocol

The primary experiment used CAUCAFall [8, 9] and GMDCSA-24 [10, 11]. CAUCAFall includes ten participants performing five daily activities and five fall types in a home environment; GMDCSA-24 adds four subject groups. Le2i [1, 12] provides a separate baseline comparison and an external test of the primary models. Table I summarizes the 450 videos. Temporal labels follow the corresponding annotations in OmniFall [13].

TABLE I: Data used for temporal fall detection. Positive windows contain the dynamic fall transition.

Dataset	Videos	Windows	Evaluation role
CAUCAFall	100	7,869	Four subject-disjoint folds
GMDCSA-24	160		
Le2i	190	12,650	Separate benchmark and external test

A pretrained YOLOv8s-pose detector [7] and BoT-SORT tracker extract the primary person trajectory. The primary comparison uses 12 body joints with normalized coordinates and confidence, giving 36 features per frame. The separate Le2i

baseline comparison uses only coordinates, giving 24 features. Both use the window construction defined in Section II.

A window is positive when at least half of its duration overlaps a fall interval and negative when no fall interval overlaps it; intermediate overlaps are excluded. For videos containing a fall, windows starting at or after the end of the final fall transition are excluded. The primary data contain 1,371 positive and 6,498 negative windows; Le2i contains 1,338 positive and 11,312 negative windows.

Four subject-disjoint folds separate the 14 primary participant groups into training, validation, and test partitions, with both datasets represented in every partition. Overlapping windows from one video remain in the same partition. Le2i is held out when training the primary models. The separate Le2i comparison uses three video-grouped folds and a 15% validation subset.

The edge deployment test used OmniFall-Synthetic [13]. Before inference, 20 fall and 20 non-fall clips were sampled with a fixed seed from the supplied annotations. These 40 clips contain 3,240 frames. The classifier was selected using primary validation F1 before synthetic evaluation. Neither the classifier nor its decision threshold was fitted to these clips. Clip-level prediction uses the maximum window probability at threshold 0.5. Clips without valid pose windows are reported separately as missing outputs.

B. Experimental environment and configuration

Table II lists the hardware and software used for model experiments. Training uses CUDA automatic mixed precision; validation and test inference use single precision.

TABLE II: Environment for the model experiments.

Component	Configuration
Operating system	Linux 5.15, x86_64
GPU	NVIDIA GeForce RTX 3090
Python	3.10.8
PyTorch	2.1.2+cu121
CUDA runtime	12.1
Training precision	Automatic mixed precision

1) *Training and comparison models*: All models train for 100 epochs using Adam, learning rate 10^{-4} , and batch size 64. The focal-loss parameter in Eq. (11) is $\gamma = 3$. Class weights are computed from the training partition for the primary comparison and set to $[1, 30]$ for the separate Le2i comparison. The primary runs clip the gradient norm at 1.0. Model selection uses validation F1. Primary experiments use seeds 42, 3407, and 2026, giving 12 evaluations per model across four folds; the Le2i comparison uses seed 42. We report arithmetic means and sample standard deviations across folds and seeds. These are descriptive summaries; no paired significance test was prespecified.

The comparison includes TCN, Transformer encoder (TE), TCNTE [1], and UniMamba. The TCN has three residual blocks with channels $[8, 16, 32]$, kernel size 5, dilations 1, 2, 4, and dropout 0.2. TE uses a 32-dimensional embedding, learned

positions, one four-head encoder layer, and feed-forward width 32. TCNTE applies that encoder to the TCN sequence. UniMamba uses the same 64-dimensional Mamba backbone as MaskedBiMamba with two forward layers and mean pooling. All primary models receive the same windows and pose features.

2) *Edge deployment configuration*: The edge platform is a 16-GB Raspberry Pi 5 with a Hailo-10H accelerator, Raspberry Pi OS (64-bit), Python 3.13, HailoRT 5.1.1, and ONNX Runtime 1.30.0. The paired timing test compares CPU ONNX and the Hailo compiled YOLOv8s-pose backend at 640×640 , using four clips replayed twice per backend after five warm-up frames. Both execute the same local temporal classifier. Inference runs inside a network namespace with no external interface or route; an outbound connection check must fail before evaluation.

3) *Evaluation metrics*: Let $y_i \in \{0, 1\}$ be the label of window i , with fall as the positive class, and let $p_i = \pi_{i,1}$ be its predicted fall probability. The classification rule is $\hat{y}_i = \mathbf{1}[p_i \geq \tau]$, where $\tau = 0.5$. Using the resulting true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), we compute

$$\begin{aligned} \text{Acc} &= \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \\ \text{Prec} &= \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Rec} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \\ \text{Spec} &= \frac{\text{TN}}{\text{TN} + \text{FP}}, \quad \text{F1} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}. \end{aligned} \quad (13)$$

Precision measures the reliability of positive predictions, recall measures fall coverage, and specificity measures rejection of non-fall activity. F1 balances precision and recall. A zero denominator gives a score of zero for these metrics.

AUROC measures probability ranking across thresholds. For N_+ positive and N_- negative windows,

$$\text{AUROC} = \frac{1}{N_+ N_-} \sum_{i: y_i=1} \sum_{j: y_j=0} \psi(p_i - p_j), \quad (14)$$

where $\psi(d)$ equals 1 for $d > 0$, $1/2$ for $d = 0$, and 0 otherwise. AUROC is undefined if either class is absent.

A video is positive when at least one of its windows is positive. If $\mathcal{W}_v$ contains the windows of video v , its probability and label are

$$p_v = \max_{i \in \mathcal{W}_v} p_i, \quad y_v = \max_{i \in \mathcal{W}_v} y_i. \quad (15)$$

Applying Eq. (13) to $(y_v, \mathbf{1}[p_v \geq \tau])$ gives video-level metrics.

Event evaluation measures temporal localization. For each video, we merge overlapping or touching positive target windows into reference intervals $\mathcal{G}_v$ and positive predicted windows into intervals $\mathcal{P}_v$. Intervals are processed in start-time order; each reference interval is paired with the first unused prediction having positive-duration overlap. Let $\mathcal{M}_v$ contain these matched pairs. Event recall is

$$R_{\text{event}} = \frac{\sum_v |\mathcal{M}_v|}{\sum_v |\mathcal{G}_v|}. \quad (16)$$

For window boundaries $[a_i, b_i]$, the evaluated duration and false-alarm rate are

$$T_{\text{eval}} = \frac{1}{3600} \sum_v \left(\max_{i \in \mathcal{W}_v} b_i - \min_{i \in \mathcal{W}_v} a_i \right), \quad (17)$$

$$\text{FA/h} = \frac{\sum_v (|\mathcal{P}_v| - |\mathcal{M}_v|)}{T_{\text{eval}}}.$$

The exposure includes the retained portions of both fall and non-fall videos. Consequently, this rate describes unmatched predicted events in the evaluation segments, not notifications during continuous monitoring. For a matched pair (g, h) with interval start times s_g and s_h , the mean onset offset is

$$D_{\text{onset}} = \frac{\sum_v \sum_{(g,h) \in \mathcal{M}_v} (s_h - s_g)}{\sum_v |\mathcal{M}_v|}. \quad (18)$$

A negative offset denotes an early prediction. This measure describes interval alignment; system latency additionally includes image processing, communication, and alert confirmation.

We also replay the deployed alert rules on each Masked-BiMamba test sequence, resetting the state between videos. A confirmed alert is matched to the first unused reference interval having positive-duration overlap with its confirmation window. Confirmed-event recall and unmatched confirmed alerts per evaluated hour use the same reference events and exposure as Eqs. (16)–(17). Confirmation offset is measured from the start of the merged positive-window interval to the end of the confirmation window. This retrospective replay evaluates fixed implementation rules; it does not tune their thresholds.

C. Results and analysis

1) *Comparison with temporal baselines:* TE achieved the highest window F1 (0.6581), AUROC (0.8881), video F1 (0.7686), and event recall (0.9767) in the primary comparison (Figure 4). At threshold 0.5, MaskedBiMamba achieved the highest window precision (0.6048) and lowest unmatched predicted-event rate (198.5 per evaluated hour). Its video F1 was 0.7632 under the same evaluation protocol. TE detected more fall windows, whereas MaskedBiMamba produced fewer unmatched events at this threshold. The comparison does not establish that one model would retain the same false-alarm ordering after model-specific calibration.

2) *Le2i baseline comparison:* Figure 4 compares the three baseline architectures on Le2i. TE achieved the highest window F1 and AUROC among the three Le2i baselines. TCNTE achieved the highest video F1 (0.8768), compared with 0.8533 for TE. Its unmatched predicted-event rate was also lower, at 118.4 versus 158.1 per evaluated hour. The change in model ranking between window and video metrics reflects the effect of temporal aggregation.

3) *Ablation study:* We evaluated ten MaskedBiMamba variants over the same four folds and three seeds (Figure 5). Removing bidirectionality reduced window F1 from 0.6395 to 0.6236; removing temporal attention reduced it to 0.6230. Both variants also reduced video F1 and increased unmatched

predicted events. These matched comparisons support retaining attention and bidirectional processing in the evaluated architecture.

The remaining components had mixed effects under the clean evaluation protocol. Removing frame masking yielded the highest clean window F1 (0.6516), while removing quality pooling yielded the highest video F1 (0.7729). Removing confidence input increased unmatched predicted events from 198.5 to 222.0 per evaluated hour. Mask ratios 0.05, 0.20, and 0.30 produced fewer unmatched events than 0.15, whereas 0.10 produced more. The model’s operating point therefore depends on how masking and quality weighting balance sensitivity against false positives.

4) *Robustness to missing observations:* We evaluated frame removal and confidence noise on the extracted pose sequences. Frame removal sets all pose and box observations to zero with probabilities 0.1, 0.2, and 0.3 while retaining the time positions. Confidence noise adds Gaussian noise with standard deviation 0.1, 0.2, or 0.3 and clips the result to $[0, 1]$. Figure 6 reports both conditions for MaskedBiMamba and TCNTE. At frame-drop probability 0.30, MaskedBiMamba retained window F1 0.6276, compared with 0.4361 for TCNTE. Their corresponding clean scores were 0.6395 and 0.6255, respectively. Confidence noise changed both models less than frame removal over the evaluated range.

5) *Cross-dataset evaluation:* We applied the trained MaskedBiMamba and TCNTE models to all 12,650 Le2i windows (Figure 6). TCNTE achieved higher clean window F1, at 0.4950 versus 0.4687 for MaskedBiMamba. Their clean video F1 values were close, at 0.8307 and 0.8299, respectively. At frame-drop probability 0.30, MaskedBiMamba retained window F1 0.4507 and video F1 0.8243, compared with TCNTE’s 0.2782 and 0.8126.

Le2i contains 130 positive and 60 negative videos under the evaluation labels. An all-positive video classifier therefore has $F1\ 260/(260 + 60) = 0.8125$. Clean video specificity is 0.1958 for MaskedBiMamba and 0.1736 for TCNTE. The high proportion of fall videos and max pooling in Eq. (15) raise video F1 here, while window F1 and video specificity give a more complete view of cross-dataset performance.

6) *Alert-state replay:* Across the 12 MaskedBiMamba evaluations, replaying the deployed confirmation rules yielded confirmed-event recall 0.7752 ± 0.0958 and 66.7 ± 45.2 unmatched confirmed alerts per evaluated hour. The mean confirmation offset was 1.88 ± 0.28 s from the start of the merged positive-window interval. These values are descriptive results for the retained test segments and fixed implementation thresholds; they are not estimates of caregiver notification burden in continuous operation.

7) *Offline edge deployment:* Figure 7 summarizes hardware measurements. The Hailo pipeline processed 20.56 ± 0.39 frames/s, compared with 1.73 ± 0.02 frames/s for CPU ONNX, an 11.9-fold throughput increase. The mean per-replay median pose latency fell from 527.1 ± 2.5 to 37.6 ± 0.4 ms. Local MaskedBiMamba inference took approximately 7.7 ms per window with the Hailo pipeline. These throughput mea-

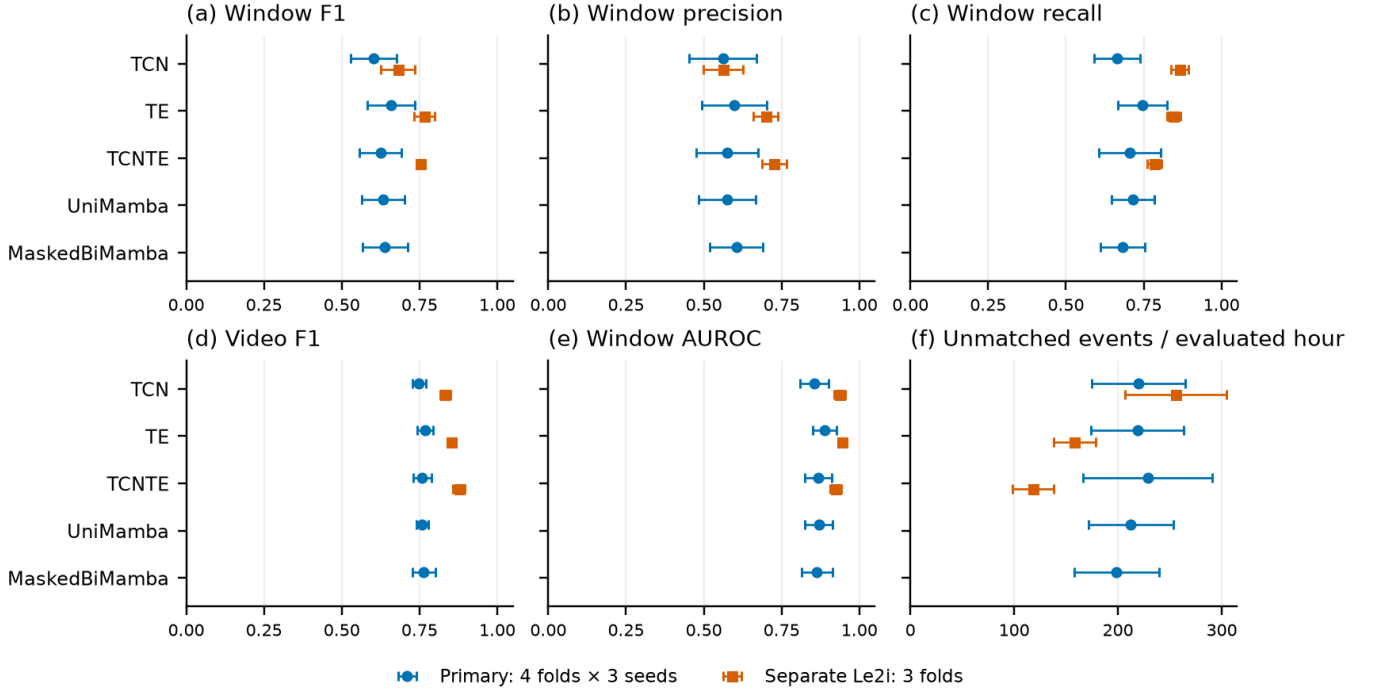


Figure 4: Temporal-baseline comparison. Points and whiskers show mean and sample standard deviation. The primary comparison uses 12 evaluations per model; the separate Le2i comparison uses three folds and is shown only for TCN, TE, and TCNTE. The primary and Le2i protocols are reported separately.

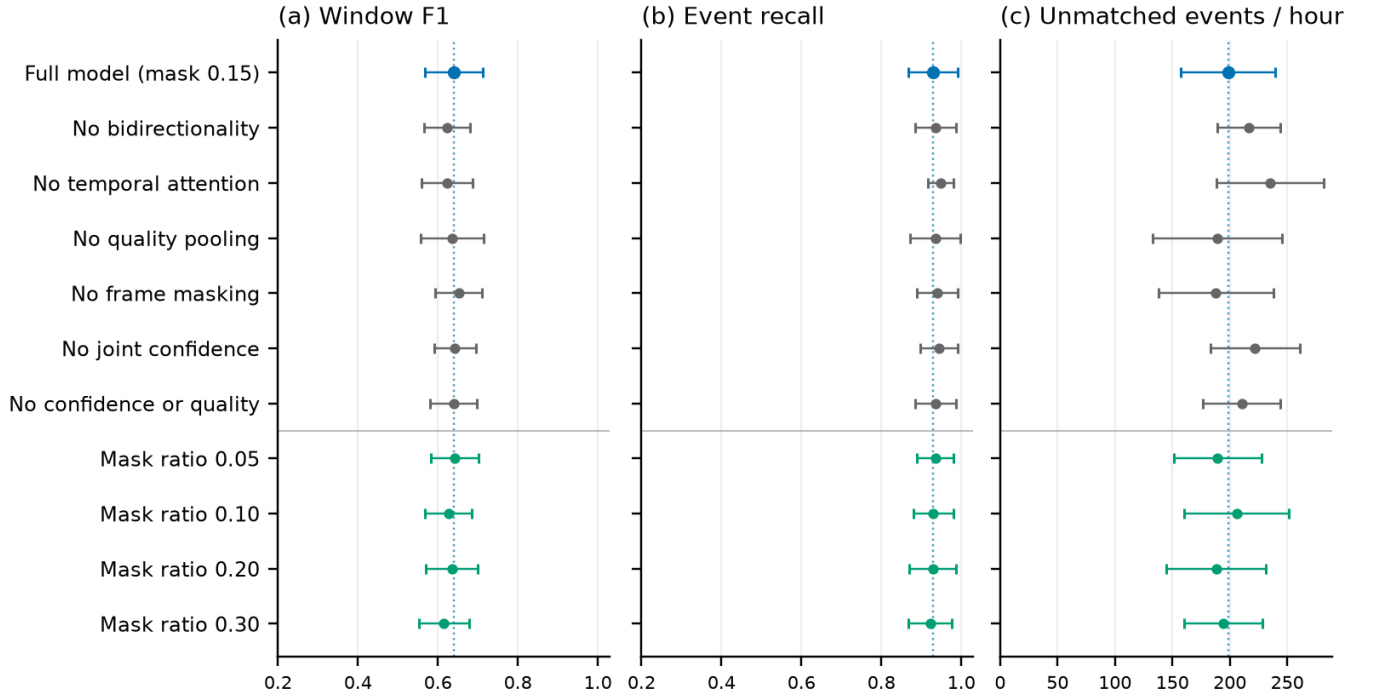


Figure 5: Ablation results across four folds and three seeds. Points and whiskers show mean and sample standard deviation. The dotted line marks the full model; green points vary the training-time frame-mask ratio. Lower unmatched-event rates are preferable.

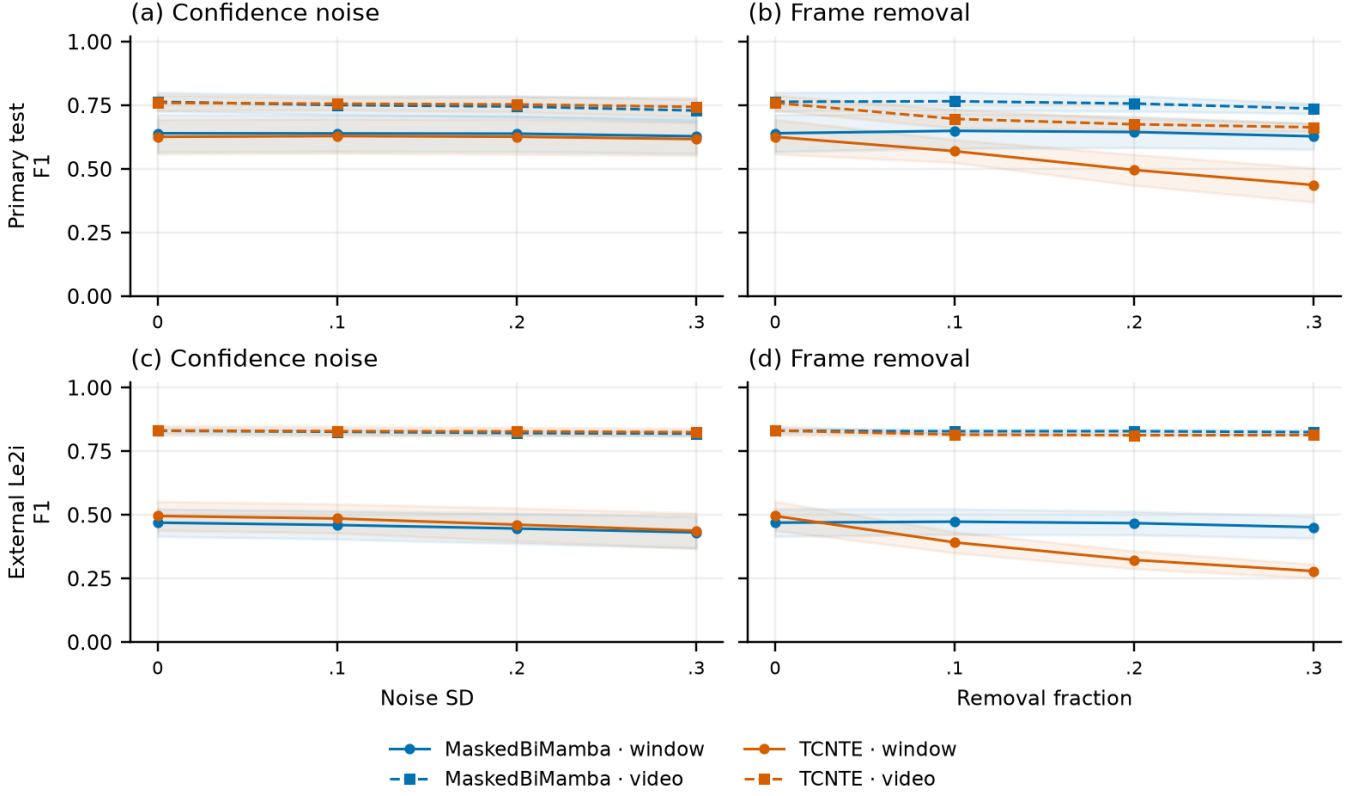


Figure 6: Robustness on the primary test partitions (top) and external Le2i videos (bottom). Blue denotes MaskedBiMamba and orange denotes TCNTE; solid lines show window F1 and dashed lines show video F1. Shading denotes one sample standard deviation over 12 evaluations. Corruptions are applied at test time.

surements include video decoding, pose processing, window classification, and local record writes, but exclude model initialization.

All 20 synthetic fall clips yielded a positive prediction, while eight of the 20 non-fall clips were false positives. One non-fall clip yielded no valid pose window for classification. Coverage is 39/40 (97.5%), accuracy among classified clips is 31/39 (79.49%), and F1 is 0.8333. Valid body poses occurred in 3,154 of 3,240 frames, producing 663 classified windows and durable records. The eight false positives include two sitting and two standing-up clips. Only 14 of the 20 positive clips produced a positive window ending inside the annotated fall interval; five of the six misses have intervals ending before the one-second input window is available. Clip-level recall therefore differs from detection during the annotated event.

A 603.8-second isolated replay processed 11,016 frames and wrote 2,312 records without interruption. CPU temperature reached 57.85°C with no reported throttling; peak process memory is 150.53 MiB. Killing the record writer preserved all 100 confirmed writes and the corresponding alert states on restart.

Six integration tests cover local storage and batch synchronization. On the deployed cloud, 34 queued records survived a withheld acknowledgement and service restart without du-

plicate insertion. Figure 8 shows delivery after a 30-second upload-path outage and the receipt-time distribution during connected operation. All 17 retained records were delivered; the queue was first observed empty 2.83 seconds after recovery. All 85 records across the delivery tests matched their edge identifiers and capture timestamps. Connected operation had a median receipt time of 1.45 seconds over 34 records (95th percentile 2.69 seconds). These measurements support recovery of the event history after interrupted connectivity. A separate USB-camera test confirmed acquisition without an external network connection. Classification from that input still requires valid full-body pose observations.

V. DISCUSSION

The comparison distinguishes the temporal representation from its decision threshold and alert confirmation rules. MaskedBiMamba achieved higher precision at the common threshold and retained more performance under frame removal than TCNTE. TE favored clean-data sensitivity, and model-specific calibration could change the ranking of unmatched-event rates. Confirmation further reduced unmatched events while lowering event recall. Calibration for a monitoring setting therefore needs to address the complete decision pipeline.

The component ablations support retaining bidirectional processing and temporal attention in this architecture. Masking

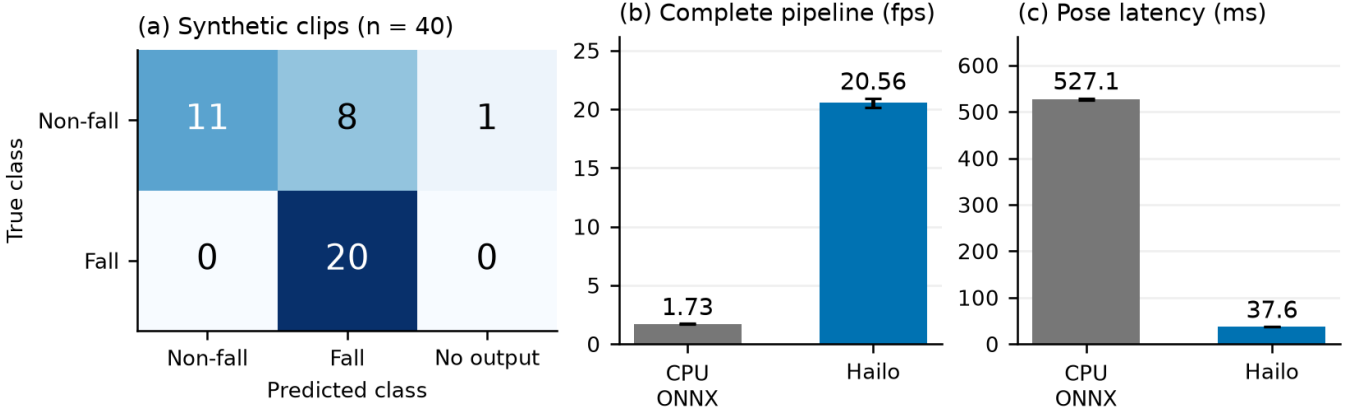


Figure 7: Raspberry Pi offline evaluation. (a) Clip-level predictions on 20 fall and 20 non-fall synthetic videos. (b,c) Mean and sample standard deviation across eight replay measurements per backend (four clips, two repetitions). Both backends use YOLOv8s-pose at 640×640 and the same local MaskedBiMamba classifier.

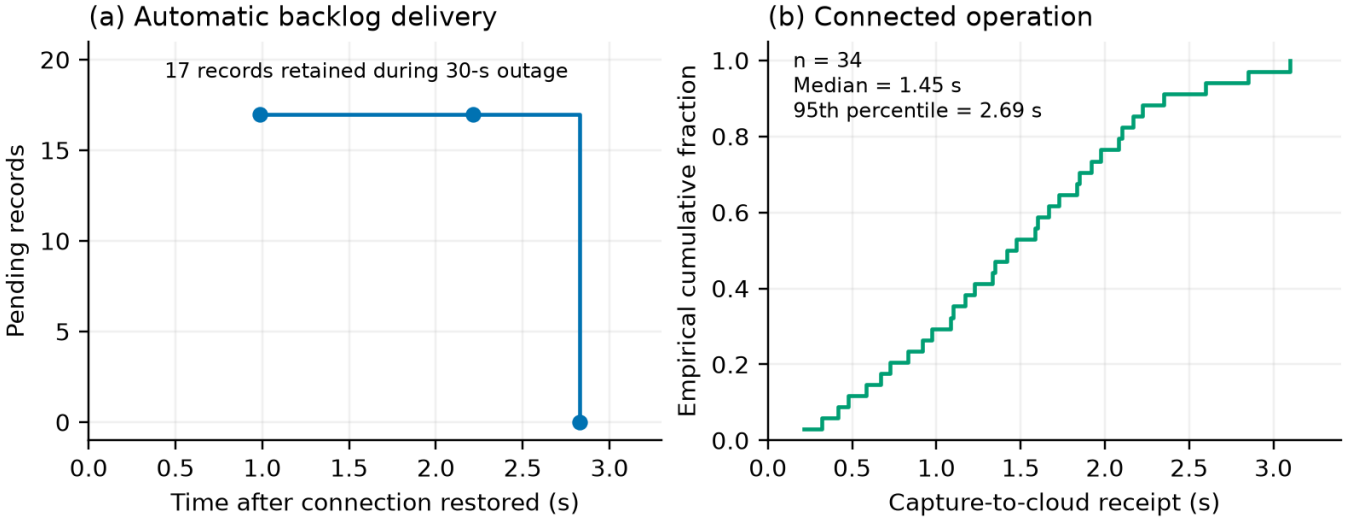


Figure 8: Cloud delivery on the deployed system. (a) Pending records sampled after reconnection; markers denote observations, with the queue first observed empty at 2.83 s. (b) Empirical distribution for 34 records during connected operation. Receipt time is measured from capture of the final frame in the classification window to cloud receipt, rather than from fall onset.

and quality pooling instead changed the sensitivity-precision balance without consistently improving clean F1. Their value depends on the observation conditions for which the classifier is calibrated. The combination extends masked temporal modeling [3] to pose sequences. However, clean-input ablations do not isolate the component responsible for frame-loss tolerance; that attribution requires perturbation-specific ablations.

Local inference separates the availability of classification from that of network connectivity. Persistent records and acknowledgement after cloud storage preserved the event history through the tested interruptions. Hailo accelerated pose extraction, while the temporal classifier remained on the Pi CPU. Throughput alone does not establish timely fall detection, which also depends on window startup, valid poses,

and alert confirmation.

Relative to the proposal, the final design moves temporal classification from the cloud to the Raspberry Pi and replaces MQTT/PostgreSQL with HTTP/SQLite. This change enables local inference and durable backlog delivery while retaining the planned cloud event history and dashboard. The proposed skeleton/geometry fusion developed into a temporal model with pose-quality features and component ablations. Offline replay and recovery tests verify the implemented data path; long-duration care monitoring remains outside the evaluated scope.

A. Limitations

The primary evaluation covers 14 subject groups, comprising ten from CAUCAFall and four from GMDCSA-24. This

limits the range of participants and fall behaviors represented. Automatically extracted poses include tracking and localization errors, while the labeling rule excludes windows with ambiguous transition overlap. Neither threshold 0.5 nor the deployed confirmation thresholds were calibrated to a particular monitoring setting. Both unmatched-event rates describe retained video segments, so they cannot estimate notification burden during continuous care.

External Le2i video F1 lies close to the all-positive baseline of 0.8125. Class imbalance and low video specificity therefore limit the evidence for cross-dataset discrimination. Reliable participant identities are unavailable for this evaluation, leaving person-independent transfer unverified.

The hardware experiment uses a small, balanced sample of synthetic videos. It establishes local execution under network isolation, but accuracy during real continuous monitoring remains untested. Changes in illumination, camera placement, and the number of people could affect pose extraction. The ten-minute replay does not establish all-day camera reliability, and power consumption remains unmeasured. Security controls were checked functionally, without penetration testing or a formal privacy analysis.

VI. CONCLUSION

MaskedBiMamba combines bidirectional temporal modeling with pose-quality weighting, and FallGuard integrates the classifier with persistent edge execution and deferred cloud delivery. Matched comparisons showed higher precision at the common threshold and greater frame-loss tolerance than TC-NTE, while TE led clean F1. Component ablations supported attention and bidirectional processing but qualified the clean-data benefits of masking and quality pooling. Hardware and recovery tests established disconnected execution and preservation of the tested records. Continuous-recording calibration and longer camera tests are needed to assess operational reliability.

REFERENCES

- [1] X. Yu, C. Wang, W. Wu, and S. Xiong, "A real-time skeleton-based fall detection algorithm based on temporal convolutional networks and transformer encoder," *Pervasive and Mobile Computing*, vol. 107, p. 102016, 2025.
- [2] D. Kim, X. Yu, and S. Xiong, "Robust skeleton-based AI for automatic multi-person fall detection on construction sites with occlusions," *Automation in Construction*, vol. 175, p. 106216, 2025.
- [3] X. Zhang, Q. Xu, F. Feng, X. Lu, and L. Xu, "Fall-Mamba: A multimodal fusion and masked mamba-based approach for fall detection," *IEEE Internet of Things Journal*, vol. 12, no. 8, pp. 10 493–10 505, 2025.
- [4] S. Chun, S. Song, D. Lee, and S. Lee, "StoneGAT: A robust fall detection framework via skeleton-aware graph attention networks," *International Journal of Control, Automation, and Systems*, vol. 23, no. 12, pp. 3702–3713, 2025.
- [5] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023. [Online]. Available: <https://arxiv.org/abs/2312.00752>
- [6] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," 2017. [Online]. Available: <https://arxiv.org/abs/1708.02002>
- [7] Ultralytics, "Pose estimation with ultralytics yolo," software documentation; accessed: 2026-09-19. [Online]. Available: <https://docs.ultralytics.com/tasks/pose/>
- [8] J. C. E. Guerrero, E. M. España, M. M. Añasco, and J. E. P. Lopera, "Dataset for human fall recognition in an uncontrolled environment," *Data in Brief*, vol. 45, p. 108610, 2022.
- [9] J. C. Eraso, E. Muñoz, M. Muñoz, and J. Pinto, "Dataset CAUCAFall," 2025, CC BY 4.0; accessed: 2026-09-17. [Online]. Available: <https://data.mendeley.com/datasets/7w7fccy7ky/5>
- [10] E. Alam, A. Sufian, P. Dutta, M. Leo, and I. A. Hameed, "GMDCSA-24: A dataset for human fall detection in videos," *Data in Brief*, vol. 57, p. 110892, 2024.
- [11] E. Alam, "Gmdcsa24: A dataset for human fall detection in videos," 2024, versioned public dataset; accessed: 2026-09-17. [Online]. Available: <https://doi.org/10.5281/zenodo.12921216>
- [12] I. Charfi, J. Mitéran, J. Dubois, M. Atri, and R. Tourki, "Definition and performance evaluation of a robust SVM-based fall detection solution," in *2012 Eighth International Conference on Signal Image Technology and Internet Based Systems*, 2012, pp. 218–224.
- [13] D. Schneider, Z. Marinov, R. Baur, Z. Zhong, R. Düger, and R. Stiefelhagen, "OmniFall: A Unified Benchmark for Staged-to-Wild Fall Detection," 2026, CC BY-NC 4.0; accessed: 2026-09-17. [Online]. Available: <https://huggingface.co/datasets/simplexsigil2/omnifall>

APPENDIX

INDIVIDUAL CONTRIBUTIONS AND REFLECTIONS

This appendix records individual contributions to FallGuard. Yaorong Huang’s entry describes his code, writing, and web-presentation responsibilities. The other entries remain TBD and must be completed by the corresponding members before submission. Author order does not imply a verified division of work.

A. Yaorong Huang

Code implementation and integration: I carried out most of the project’s code implementation and organization, covering temporal modeling, pose preprocessing, training/evaluation tools, model export, and edge–cloud integration. MaskedBiMamba combines established Mamba operators and pose estimation with temporal attention, bidirectional processing, frame masking, and quality weighting. My contribution concerns their composition, implementation, evaluation tools, and integration; the underlying operators and pose extractor originate from the cited work.

Public evidence includes `code/src/fallbench/` and `code/scripts/export_mamba_onnx.py`. The released `deployment/` directory contains the offline runtime, independent outbox, cloud puller, dashboards, and generic service templates. Selected temporal weights and checksums are provided in `artifacts/models/`.

Offline deployment and workspaces: I integrated USB-camera/video acquisition, Hailo pose extraction with a CPU fallback, and the two-input MaskedBiMamba ONNX classifier on the Pi. Separating inference from the SQLite outbox preserves committed records during disconnection or camera stoppage. Cloud synchronization commits records before acknowledging UUIDs and retains original capture times during retry.

I completed the three-page local and cloud workspaces: live preview and fresh results, historical charts with filters/pagination/CSV, and device diagnostics. The earlier minimal local page provided insufficient independent observation and troubleshooting. The updated interface clears current probability for missing poses, expired results, or lost connections. The cloud retains its archive while current Pi status uses a protected private bridge; controls validate same-origin JSON and a page token.

Deployment verification preserved existing databases, checked service health and source hashes, and saved rollback backups. Temporary integration tests cover archive queries, restart persistence, authentication, delivery, and actual service startup/shutdown; both actual page scripts were checked for stale and invalid states. Public deployment evidence is in `results/deployment/`.

Technical issues and evidence boundaries: Archived missing-joint tests zero-filled coordinates before normalization, also rescaling observed joints. I corrected the order while excluding the affected results from current conclusions; the fix does not validate those old experiments. I also separated window/video predictions, merged events, and fixed-

rule confirmed alerts. Pipeline throughput, detection timing, cloud receipt latency, and operational archive counts describe different quantities and do not establish the same performance guarantee.

Manuscript and evidence organization: I took primary responsibility for preparing and revising the LaTeX manuscript, method notation, protocols, results, implementation, and limitations. I maintained consistency across figures, aggregate evidence, technical explanations, deployed services, and repository materials. The interpretation retains TE’s higher clean F1, mixed ablations, limited external discrimination, and incomplete fall-interval detection. Evidence is retained in `manuscripts/final-report/` and `results/`; external algorithms and datasets are credited.

Presentation and publication: I developed the twelve-slide GitHub Pages presentation in `docs/`, including interactive charts, bilingual notes, searchable results, projection controls, a presenter console, and rehearsal timing. I synchronized the final report, Chinese guides, presentation, deployment source, and GitHub publication with the October workspace update. The presentation exposes saved evidence; the Pi and cloud provide the separate live demonstration.

Reflection: A working prototype and a supported performance claim need different evidence. High clip recall can coexist with late detection; high video F1 can coexist with low specificity. Preprocessing can invalidate a robustness comparison. Deployment also requires freshness checks, independent storage, recovery, diagnostics, and consistent documentation. Future work should evaluate continuous recordings, calibrate alerts, and test longer camera operation across conditions, while preserving one traceable evidence chain.

B. Jiahao Zhao

TBD.

C. Chengyi Xu

TBD.

D. Zhonglin Chang

TBD.

E. Jiayun Song

TBD.